Before containers, you would rely on virtual machines to guarantee the reproducibility of the environment, isolation, and configurability. Virtualization works by leveraging a hypervisor component that abstracts the hardware, making it possible to run multiple operating systems on the same machine in an isolated fashion. The hypervisor would run directly on the machine hardware (type 1) or on the host operating system (type 2).

On the other hand, an *OS container* is a lightweight executable package that includes everything needed to run the application. Containers share the same kernel as the host: there's no need to bootstrap full operating systems to add new isolated contexts. On Linux, that is possible by leveraging a couple of features offered by the Linux kernel:

- *namespaces* for partitioning resources among processes so that each process (or group of processes) can only see a subset of the resources available on the machine
- *cgroups* for controlling and limiting the resource usage for a process (or group of processes)

NOTE When using virtualization only, the hardware is shared, whereas containers also share the same operating system kernel. Both provide computing environments for running software in isolation, even if the degree of isolation is not the same.

Figure 1.10 shows the difference between virtualization and container technologies.